

Embodied AI
Lab Manual for Isaac Lab/Isaac Sim
R1 Flat-Terrain/Standing Setup
August 2026

Prepared by

Mario Ortiz

Table of Contents

1	Objective.....	4
2	Purpose of the Flat Environment.....	4
3	Purpose of the Standing Environment	4
4	File Structure	5
5	Flat Environment Task	6
5.1	Create the R1 Flat Environment Folder	6
5.2	Import the Rough Environment.....	6
5.3	Create the Flat Environment Class	6
5.4	Force the Terrain to a Flat Plane	7
5.5	Remove the Height Scanner	7
5.6	Remove the Terrain Curriculum	7
5.7	Adjust Rewards for Flat Walking	8
5.8	Set Flat Walking Commands	8
5.9	Create the Flat Play Class	9
5.10	Register the Flat R1 Task	9
5.11	Add the Flat PPO Runner.....	10
5.12	Train the Flat R1 Task	10
5.13	Play the Flat R1 Policy	11
6	Create Standing Environment Task	11
6.1	Create the standing environment file	12
6.2	Import the Rough Environment.....	12
6.3	Create the Standing Environment Class	12
6.4	Force Flat Terrain and Height Scanning	12
6.5	Set Zero-Velocity Commands.....	12
6.6	Remove Walking-Specific Rewards and Adjust Rewards for Standing	13
6.7	Set the Episode Length	14
6.8	Set up the Standing Environment Play Class.....	14
6.9	Set up the Standing Environment Task for Training	14

6.10	Training and Viewing the Standing Task	16
------	--	----

1 Objective

The objective of this lab is to configure a flat-terrain locomotion environment and a standing task for the Unitree R1. This lab will introduce students to the process of developing custom locomotion tasks using Isaac Lab and Isaac Sim.

The R1's **rough_env_cfg.py** file provided for the Unitree R1 already contains essential R1-specific configuration, including:

- R1 robot asset
- R1 body and link names
- R1 contact links
- R1 fall termination conditions
- R1-specific reward modifications
- R1 terrain configuration

Rather than recreating these components from scratch, new environments can be developed by inheriting from **R1RoughEnvCfg** and modifying only the parameters relevant to the desired task. This approach promotes code reuse and provides a consistent foundation for developing additional R1 locomotion environments.

2 Purpose of the Flat Environment

The flat environment is based on the structure of the Unitree G1 **flat_env_cfg.py** configuration. However, this lab will create a corresponding flat-terrain environment for the Unitree R1.

Students will modify the inherited configuration to adapt the environment for flat terrain as well as remove parameters that are specific to the rough-terrain locomotion task.

This exercise demonstrates how inheritance can be used in Isaac Lab to efficiently create new environments while preserving configurations that have already been established.

3 Purpose of the Standing Environment

The standing environment is a simplified locomotion task in which the robot is not required to walk or turn. Instead, the commanded velocity is set to zero, and the objective is for the robot to remain upright and stable. A standing task can serve as a foundation for more complex behaviors. For example, maintaining a stable standing posture may be useful when training the robot to perform upper-body tasks while keeping its lower body stationary.

Similar to the flat environment, the standing task inherits from **R1RoughEnvCfg**. This allows it to retain important R1-specific configurations, including the robot asset, contact configuration, and fall-detection logic, while modifying the parameters necessary for the standing task.

The primary modifications include:

- Use a flat-plane terrain
- Remove the height scanner
- Command zero velocity
- Remove walking-specific rewards
- Increase the weight of standing-related rewards
- Reduce the episode length

4 File Structure

C:\isaacrab

```

├─ source
|   └─ isaacrab_tasks
|       └─ isaacrab_tasks
|           └─ manager_based
|               └─ locomotion
|                   └─ velocity
|                       └─ config
|                           └─ r1
|                               ├── __init__.py
|                               ├── r1_asset_cfg.py
|                               ├── rough_env_cfg.py
|                               ├── flat_env_cfg.py
|                               ├── stand_env_cfg.py
|                               |
|                               ├── assets
|                               |   └─ R1
|                               |       ├── R1.usd
|                               |       └─ configuration
|                               |
|                               └─ agents
|                                   ├── __init__.py
|                                   └─ rsl_r1_ppo_cfg.py

```

```

└─ logs
    └─ rs1_r1
        └─ r1_flat
            └─ YYYY-MM-DD_HH-MM-SS
                └─ model_0.pt
                └─ model_750.pt
                └─ model_1499.pt
            └─ r1_stand
                └─ YYYY-MM-DD_HH-MM-SS
                    └─ model_0.pt
                    └─ model_500.pt
                    └─ model_999.pt

```

5 Flat Environment Task

5.1 Create the R1 Flat Environment Folder

Go to the R1 configuration folder:

```

cd
C:\isaac\source\isaac\tasks\isaac\tasks\manager_based\locomotion\velocity\config\r1

```

Create and open the file:

```
code flat_env_cfg.py
```

5.2 Import the Rough Environment

At the top of flat_env_cfg.py, import R1RoughEnvCfg:

```

from isaacsim.utils import configclass
from .rough_env_cfg import R1RoughEnvCfg

```

5.3 Create the Flat Environment Class

The flat environment inherits from R1RoughEnvCfg:

```
@configclass
class R1FlatEnvCfg(R1RoughEnvCfg):
    def __post_init__(self):
        super().__post_init__()
```

The line:

```
super().__post_init__()
```

runs all the setup from the rough environment first. After that, the flat environment overrides only the parts that need to change.

5.4 Force the Terrain to a Flat Plane

The rough environment used generated terrain. The flat environment replaces that with a simple plane:

```
self.scene.terrain.terrain_type = "plane"
self.scene.terrain.terrain_generator = None
```

Since the terrain is now flat, the terrain curriculum is not needed:

```
if hasattr(self.scene.terrain, "max_init_terrain_level"):
    self.scene.terrain.max_init_terrain_level = None
```

5.5 Remove the Height Scanner

The rough environment needed the height scanner so the policy can observe terrain height changes. For flat ground, this is unnecessary.

```
self.scene.height_scanner = None
if hasattr(self.observations.policy, "height_scan"):
    self.observations.policy.height_scan = None
```

5.6 Remove the Terrain Curriculum

The terrain curriculum is only needed for generated terrain. Since this environment uses a plane, we can remove it:

```
self.scene.height_scanner = None
if hasattr(self.observations.policy, "height_scan"):
    self.observations.policy.height_scan = None
```

5.7 Adjust Rewards for Flat Walking

The flat environment keeps the R1 reward structure but modifies some values.

The most important change is adding a vertical velocity penalty:

```
if hasattr(self.rewards, "lin_vel_z_l2") and self.rewards.lin_vel_z_l2 is not None:
    self.rewards.lin_vel_z_l2.weight = -0.2
```

This helps reduce hopping on flat ground.

The yaw tracking reward is also reduced from the rough value:

```
if hasattr(self.rewards, "track_ang_vel_z_exp") and self.rewards.track_ang_vel_z_exp is not None:
    self.rewards.track_ang_vel_z_exp.weight = 1.0
```

The smoothness and effort penalties are kept like the rough environment:

```
if hasattr(self.rewards, "action_rate_l2") and self.rewards.action_rate_l2 is not None:
    self.rewards.action_rate_l2.weight = -0.005

if hasattr(self.rewards, "dof_acc_l2") and self.rewards.dof_acc_l2 is not None:
    self.rewards.dof_acc_l2.weight = -1.25e-7

if hasattr(self.rewards, "dof_torques_l2") and self.rewards.dof_torques_l2 is not None:
    self.rewards.dof_torques_l2.weight = -1.5e-7
```

5.8 Set Flat Walking Commands

The flat walking task still trains the robot to move forward and turn:

```
self.commands.base_velocity.ranges.lin_vel_x = (0.0, 1.0)
self.commands.base_velocity.ranges.lin_vel_y = (0.0, 0.0)
self.commands.base_velocity.ranges.ang_vel_z = (-1.0, 1.0)
```

This means:

Forward speed ranges from 0.0 to 1.0 m/s

Side velocity is disabled

Yaw turning ranges from -1.0 m/s to 1.0 m/s

5.9 Create the Flat Play Class

The play class is used to visually evaluate a trained policy:

```
@configclass

class R1FlatEnvCfg_PLAY(R1FlatEnvCfg):

    def __post_init__(self):
        super().__post_init__()
```

The play version uses multiple robots:

```
self.scene.num_envs = 50

self.scene.env_spacing = 2.5

self.episode_length_s = 40.0
```

It also forces flat terrain again:

```
self.scene.terrain.terrain_type = "plane"

self.scene.terrain.terrain_generator = None
```

The play command uses a fixed forward walking speed so the robot only walks forward:

```
self.commands.base_velocity.ranges.lin_vel_x = (0.5, 0.5)

self.commands.base_velocity.ranges.lin_vel_y = (0.0, 0.0)

self.commands.base_velocity.ranges.ang_vel_z = (0.0, 0.0)
```

5.10 Register the Flat R1 Task

Open:

code `__init__.py`

Add these imports to the code:

```
from .flat_env_cfg import R1FlatEnvCfg, R1FlatEnvCfg_PLAY
```

Then register the training task in the code:

```
gym.register(

    id="Isaac-Velocity-Flat-R1-v0",

    entry_point="isaac.lab.envs:ManagerBasedRLEnv",

    disable_env_checker=True,

    kwargs={
        "env_cfg_entry_point": R1FlatEnvCfg,
```

```

        "rsl_rl_cfg_entry_point": f"{agents.__name__}.rsl_rl_ppo_cfg:R1FlatPPORunnerCfg",
    },
)

```

5.11 Add the Flat PPO Runner

Open:

```
code agents\rsl_rl_ppo_cfg.py
```

Add this RL class to the code:

```

@configclass
class R1FlatPPORunnerCfg(R1RoughPPORunnerCfg):
    def __post_init__(self):
        super().__post_init__()

        self.max_iterations = 1500
        self.experiment_name = "r1_flat"
        self.policy.actor_hidden_dims = [512, 256, 128]
        self.policy.critic_hidden_dims = [512, 256, 128]

```

This tells Isaac Lab to save flat-training logs under:

```
C:\isaacclab\logs\rsl_rl\r1_flat
```

5.12 Train the Flat R1 Task

Ensure the R1 task works properly with a 1 iteration test:

```

cd C:\isaacclab
.\isaacclab.bat -p .\scripts\reinforcement_learning\rsl_rl\train.py `
    --task Isaac-Velocity-Flat-R1-v0 `
    --num_envs 4 `
    --max_iterations 1

```

Then run the full flat training:

```

cd C:\isaacclab
.\isaacclab.bat -p .\scripts\reinforcement_learning\rsl_rl\train.py `

```

```
--task Isaac-Velocity-Flat-R1-v0 `
--num_envs 1024 `
--max_iterations 1500 `
--headless
```

5.13 Play the Flat R1 Policy

Locate the newest run and then play the final checkpoint:

```
cd C:\isaaclab
.\isaaclab.bat -p .\scripts\reinforcement_learning\rsl_r1\train.py `
--task Isaac-Velocity-Flat-R1-v0 `
--num_envs 1024 `
--max_iterations 1500 `
--headless
```

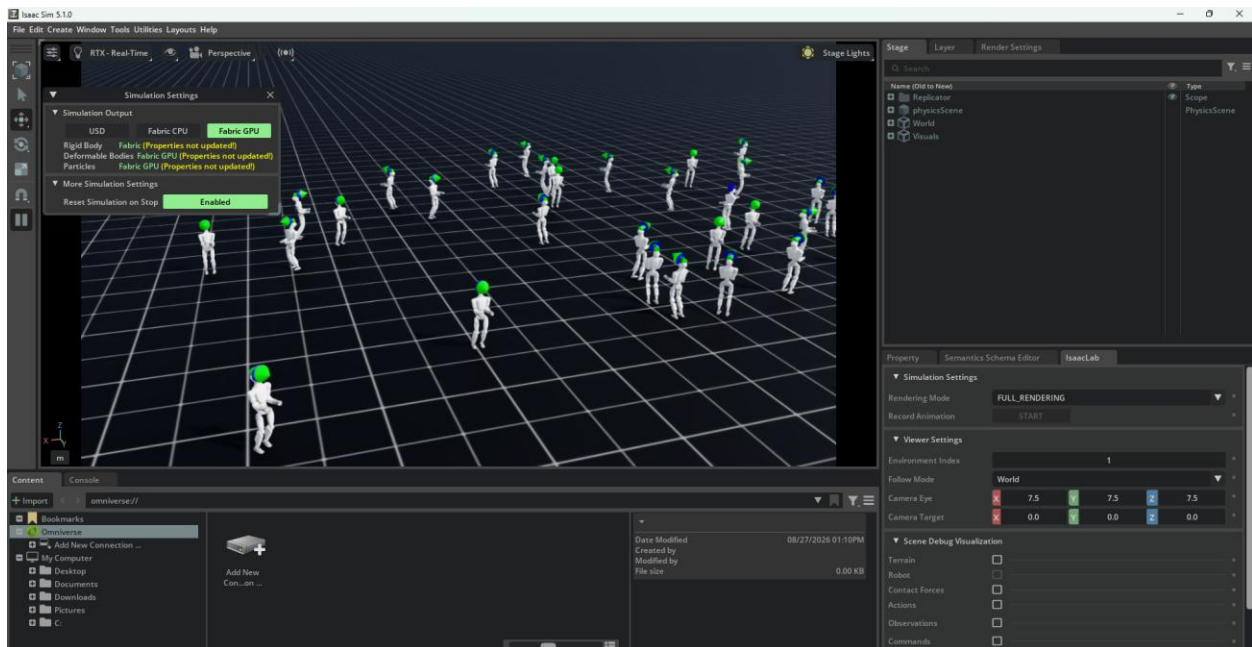


Figure 1: Isaac Sim GUI of the Unitree R1 Flat-Environment Task; Arrows indicate the direction the robots move toward to

6 Create Standing Environment Task

The imports for the standing environment are like those used for the flat environment, since this task also uses flat terrain. We will import `R1RoughEnvCfg` and use it as the base class for the standing environment. From there, we can modify the necessary parameters to configure the environment for a standing task.

6.1 Create the standing environment file

```
cd
C:\isaac\source\isaac\isaac_tasks\manager_based\locomotion\velocity\config\r1
code stand_env_cfg.py
```

6.2 Import the Rough Environment

```
from isaacsim.utils import configclass
from .rough_env_cfg import R1RoughEnvCfg
```

6.3 Create the Standing Environment Class

```
@configclass
class R1StandEnvCfg(R1RoughEnvCfg):
    def __post_init__(self):
        super().__post_init__()
```

6.4 Force Flat Terrain and Height Scanning

Similarly removing rough terrain and height scanning:

```
self.scene.terrain.terrain_type = "plane"
self.scene.terrain.terrain_generator = None

if hasattr(self.scene.terrain, "max_init_terrain_level"):
    self.scene.terrain.max_init_terrain_level = None
```

6.5 Set Zero-Velocity Commands

For standing, the command is zero motion:

```
self.commands.base_velocity.ranges.lin_vel_x = (0.0, 0.0)
self.commands.base_velocity.ranges.lin_vel_y = (0.0, 0.0)
self.commands.base_velocity.ranges.ang_vel_z = (0.0, 0.0)
```

If heading exists, set it to zero:

```
if hasattr(self.commands.base_velocity.ranges, "heading"):
```

```
self.commands.base_velocity.ranges.heading = (0.0, 0.0)
```

This changes the meaning of the velocity tracking rewards. The robot will now be rewarded for staying still.

6.6 Remove Walking-Specific Rewards and Adjust Rewards for Standing

The walking rewards are no longer needed and hence removed:

```
if hasattr(self.rewards, "feet_air_time"):
    self.rewards.feet_air_time = None

if hasattr(self.rewards, "swing_knee_flexion"):
    self.rewards.swing_knee_flexion = None

if hasattr(self.rewards, "both_feet_airborne"):
    self.rewards.both_feet_airborne = None
```

We increase this weight to 2.0 to give the reward a greater contribution to the overall reward of velocity tracking.

```
if hasattr(self.rewards, "track_lin_vel_xy_exp") and self.rewards.track_lin_vel_xy_exp is not None:
    self.rewards.track_lin_vel_xy_exp.weight = 2.0
```

Yaw tracking is kept:

```
if hasattr(self.rewards, "track_ang_vel_z_exp") and self.rewards.track_ang_vel_z_exp is not None:
    self.rewards.track_ang_vel_z_exp.weight = 1.0
```

Vertical Movement is penalized strongly:

```
if hasattr(self.rewards, "lin_vel_z_l2") and self.rewards.lin_vel_z_l2 is not None:
    self.rewards.lin_vel_z_l2.weight = -1.0
```

The upright orientation reward is strengthened:

```
if hasattr(self.rewards, "flat_orientation_l2") and self.rewards.flat_orientation_l2 is not None:
```

```
    self.rewards.flat_orientation_l2.weight = -2.0
```

Feet sliding is penalized more than in walking:

```
if hasattr(self.rewards, "feet_slide") and self.rewards.feet_slide is not None:
```

```
    self.rewards.feet_slide.weight = -0.25
```

6.7 Set the Episode Length

We can use a shorter episode length during training. The play version uses a longer episode so the user can visually inspect the robot.

```
self.episode_length_s = 20.0
```

6.8 Set up the Standing Environment Play Class

```
@configclass
```

```
class R1StandEnvCfg_PLAY(R1StandEnvCfg):
```

```
    def __post_init__(self):
```

```
        super().__post_init__()
```

```
self.scene.num_envs = 1
```

```
self.scene.env_spacing = 2.5
```

```
self.episode_length_s = 40.0
```

```
self.commands.base_velocity.ranges.lin_vel_x = (0.0, 0.0)
```

```
self.commands.base_velocity.ranges.lin_vel_y = (0.0, 0.0)
```

```
self.commands.base_velocity.ranges.ang_vel_z = (0.0, 0.0)
```

6.9 Set up the Standing Environment Task for Training

Code `__init__.py`

Add the import:

```
from .stand_env_cfg import R1StandEnvCfg, R1StandEnvCfg_PLAY
```

Register the training task and the play task:

```
gym.register(
    id="Isaac-Velocity-Flat-R1-Stand-v0",
    entry_point="isaacsim.envs:ManagerBasedRLEnv",
    disable_env_checker=True,
    kwargs={
        "env_cfg_entry_point": R1StandEnvCfg,
        "rsl_rl_cfg_entry_point": f"{agents.__name__}.rsl_rl_ppo_cfg:R1StandPPORunnerCfg",
    },
)
```

```
gym.register(
    id="Isaac-Velocity-Flat-R1-Stand-Play-v0",
    entry_point="isaacsim.envs:ManagerBasedRLEnv",
    disable_env_checker=True,
    kwargs={
        "env_cfg_entry_point": R1StandEnvCfg_PLAY,
        "rsl_rl_cfg_entry_point": f"{agents.__name__}.rsl_rl_ppo_cfg:R1StandPPORunnerCfg",
    },
)
```

Open:

code agents\rsl_rl_ppo_cfg.py

Add:

@configclass

class R1StandPPORunnerCfg(R1FlatPPORunnerCfg):

def __post_init__(self):

super().__post_init__()

self.max_iterations = 1000

self.experiment_name = "r1_stand"

```

self.policy.actor_hidden_dims = [256, 128, 128]
self.policy.critic_hidden_dims = [256, 128, 128]

self.policy.init_noise_std = 0.5
self.algorithm.entropy_coef = 0.003
self.algorithm.learning_rate = 1.0e-3

```

6.10 Training and Viewing the Standing Task

Verification Test:

```
cd C:\isaacLab
```

```

.\isaacLab.bat -p .\scripts\reinforcement_learning\rsl_rl\train.py `
  --task Isaac-Velocity-Flat-R1-Stand-v0 `
  --num_envs 4 `
  --max_iterations 1

```

Full Standing Training:

```

cd C:\isaacLab

.\isaacLab.bat -p .\scripts\reinforcement_learning\rsl_rl\train.py `
  --task Isaac-Velocity-Flat-R1-Stand-v0 `
  --num_envs 1024 `
  --max_iterations 1000 `
  --headless

```

Note that **\$r1StandPath** represents the specific folder in which your model is played in. You will find this name in the folder:

```
\isaacLab\logs\rsl_rl\
```

Play the Standing Policy:

```

.\isaacLab.bat -p .\scripts\reinforcement_learning\rsl_rl\play.py `
  --task Isaac-Velocity-Flat-R1-Stand-Play-v0 `
  --checkpoint "$r1StandPath\model_999.pt"

```

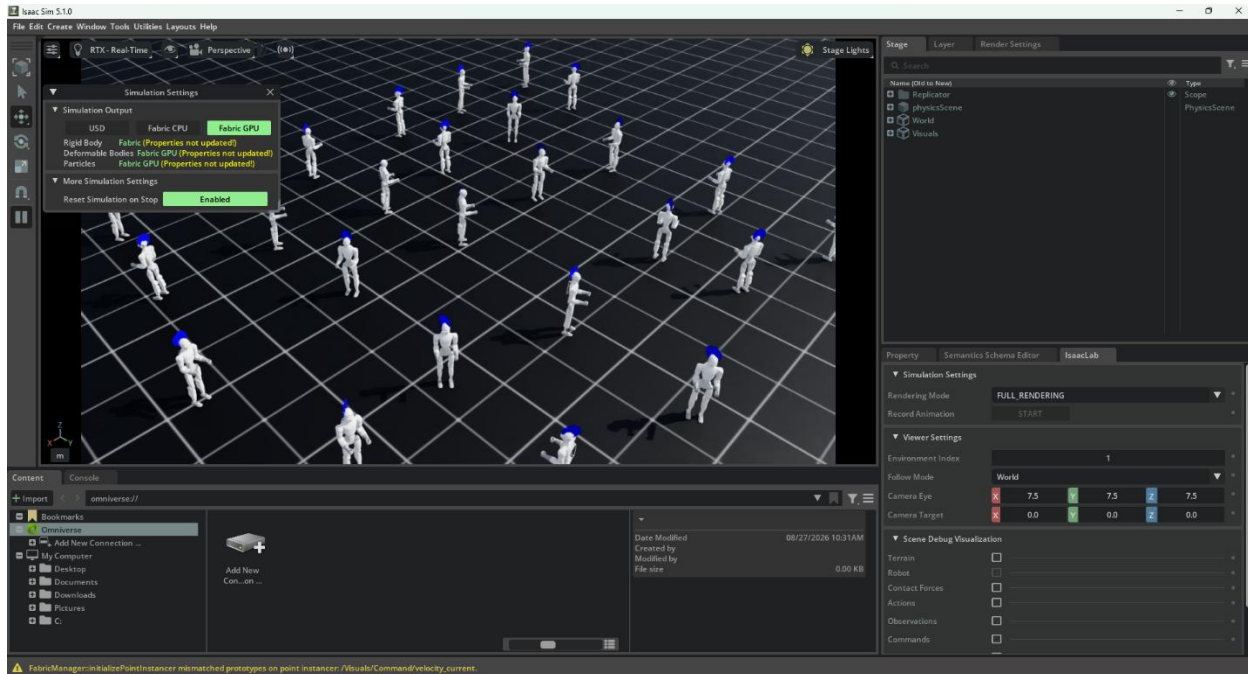



Figure 2: Isaac Sim GUI of the Unitree R1 Standing-Task; Standing/lack of motion is shown by the lack of motion shown by the circles on their heads

7 References

- [1] NVIDIA, “Creating a Manager-Based RL Environment,” Isaac Lab Documentation.
- [2] NVIDIA, “isaacsim.managers,” Isaac Lab API Documentation.
- [3] NVIDIA, “Registering an Environment,” Isaac Lab Documentation.
- [4] NVIDIA, “isaacsim_rl,” Isaac Lab RL API Documentation.
- [5] RSL-RL, “Configuration,” RSL-RL Documentation.
- [6] Gymnasium, “Register and Make,” Gymnasium Documentation.